



Qiskit Summer Dojo: Qiskit v2.X資格対策 #2 ～ 量子回路の作成実行編～

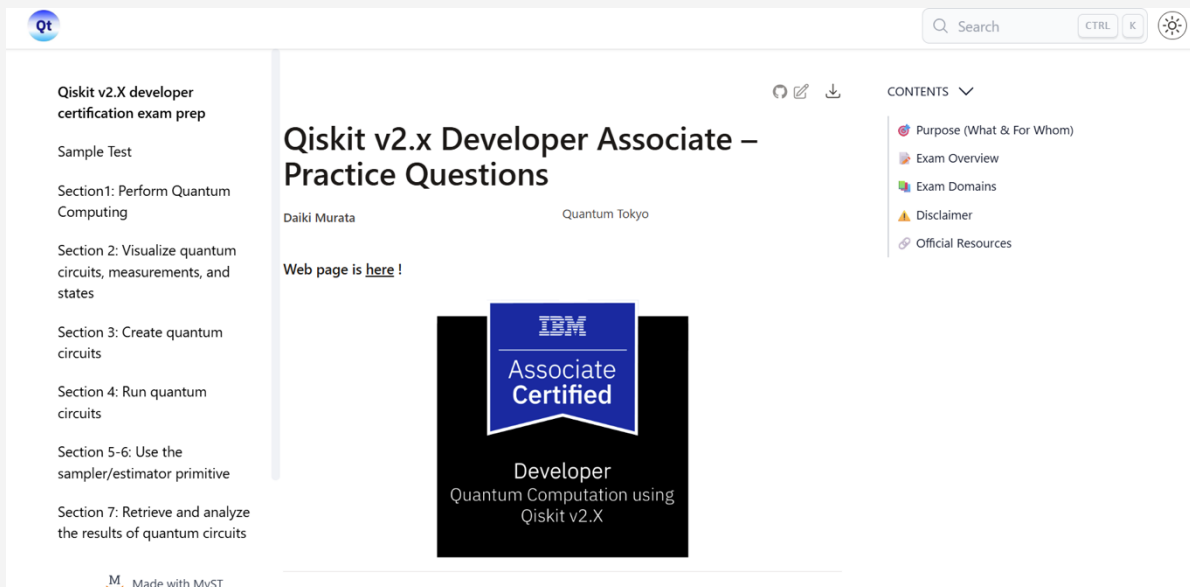
2026/08/20

Kaori Namba



「Qiskit v2.x Developer Associate – Practice Questions」に掲載されている内容について重要ポイントの紹介とそれに紐づく問題の解説を行います。

- **問題はこちら** : <https://quantum-tokyo.github.io/qiskit-certification-prep/>
- 例題を出した時に少し時間を取りますので、正解がどれかを考えてみてください。
- 本セッションでは時間の関係で全ての問題の解説は行いませんので、ご了承ください。



- **Section3: Create quantum circuits**
 - TASK 3.1: Construct dynamic circuits
 - TASK 3.2: Construct parameterized circuits (例題近日追加予定)
 - TASK 3.3: Transpile and optimize circuits
 - TASK 3.4: Construct basic quantum circuits (例題なし)
- **Section 4: Run quantum circuits**
 - TASK 4.1: Demonstrate an understanding of execution modes such as: session with dedicated, priority, and batch mode
 1. Sessions
 2. Batch
 - TASK 4.2: Demonstrate understanding of how to run quantum circuits with real hardware using Qiskit Runtime primitives and applying broadcasting rule
 1. Primitives
 2. Broadcasting rules
 3. Input / output
 4. Jobs

Section3: Create quantum circuits

TASK 3.1: Construct dynamic circuits

REFERENCES:

<https://quantum.cloud.ibm.com/docs/en/guides/classical-feedforward-and-control-flow>



TASK 3.1: Overview

Dynamic circuit = Classical feedforward

量子回路の実行中に中間測定を行い、その結果に基づいて古典的なロジックを回路内で実行する仕組み。

- 用途: 量子状態(GHZ, W, 行列積, etc.)の準備、長距離ビットのエンタングルメント、IQP* 的回路のサンプリング、の効率化
- 長所: 回路が浅くなり、エラーが低減される
- 短所: 回路の中間測定や古典的な演算は、通常2量子ビットゲートよりも実行時間が長くなる

if_test

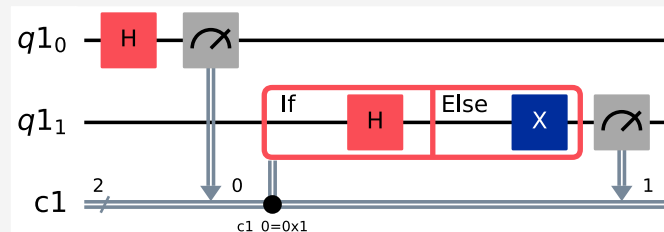
- with文と使用

```
circuit.h(q0)
circuit.measure(q0, c0)
with circuit.if_test((c0, 1)) as else_:
    circuit.h(q1)
with else_:
    circuit.x(q1)
circuit.measure(q1, c1)
```

- 複数ビットを評価したい場合は2進数リテラルを活用
`with circuit.if_test((clbits, 0b001)):`
- 制限: ネストできない、if_test内でreset/measure不可

制御フロー文

Qiskit	QASM	IQCS**
if_test	if	Supported
switch	switch	
for	for	
while	while	



* ... Instantaneous Quantum Polynomial-time

** ... IBM Quantum Compute Service



TASK 3.1: Overview

Broadcast

- 制御フローの条件を評価するために、測定データを制御ロジックに転送する
- 数え方
 - 同じ古典ビット（変化なし）を複数の `if_test` で使う → 1 broadcast
 - 別の古典レジスタ（または測定後に値が変わったビット）を使う → 新たな broadcast
- 制限
 - 1回の broadcast で転送できるのは 最大 60 bit
 - `if_test` のオペランドは 最大 32 bit の古典レジスタのみ
 - `barrier()` を挟むと broadcast が分割される → 60 bit 超えを避ける手段

Classical Expression

- 回路実行中に古典値を演算。 `qiskit.circuit.classical.expr` モジュールに実装
- `qiskit.circuit.Store` は繰り返し使う古典式の結果を保存可能。演算が自動的に並列化されるため、実行時の効率向上

関数	説明
<code>lift(x)</code>	任意のオブジェクトを古典式に変換
<code>bit_xor(a, b)</code>	ビット単位 XOR
<code>bit_not(a)</code>	ビット単位 NOT
<code>index(reg, i)</code>	レジスタの <i>i</i> 番目のビットを参照

3. How do you create an `else` block paired with an `if_test` block?
 - a. Chain another `with circuit.if_test((c, 0)):` immediately after
 - b. Capture the context manager as a target in `with circuit.if_test(...) as else_:` and then use `with else_:` for the `else` block
 - c. Use `with circuit.else_test():` right after the `if` block
 - d. Define an `else:` statement in regular Python (outside the circuit) to emit circuit ops

TASK 3.1: 例題

3. How do you create an `else` block paired with an `if_test` block?

a. Chain another `with circuit.if_test((c, 0)):` immediately after

b. Capture the context manager as a target in `with circuit.if_test(...) as else_:` and then use `with else_:` for the `else` block

c. Use `with circuit.else_test():` right after the `if` block

d. Define an `else:` statement in regular Python (outside the circuit) to emit circuit ops

11. In a job, you measure into `cr1` (32 bits) and `cr2` (32 bits), then immediately test both registers in two `if_test` statements without a barrier. Why can this be invalid?
- a. Two separate `if_test` blocks are illegal in the same circuit
 - b. The total broadcasted bits (32 + 32) can exceed the 60-bit limit in one broadcast group
 - c. Measuring more than 32 qubits in one job is not supported
 - d. The `barrier` instruction is deprecated in dynamic circuits

TASK 3.1: 例題

11. In a job, you measure into **cr1** (32 bits) and **cr2** (32 bits), then immediately test both registers in two **if_test** statements without a barrier. Why can this be invalid?

- a. Two separate **if_test** blocks are illegal in the same circuit
- b.** The total broadcasted bits (32 + 32) can exceed the 60-bit limit in one broadcast group
- c. Measuring more than 32 qubits in one job is not supported
- d. The **barrier** instruction is deprecated in dynamic circuits

```
# cr1=32bit, cr2=32bit を barrier なしで連続テスト
qc.measure(range(32), cr1)
qc.measure(range(32, 64), cr2)
with qc.if_test((cr1, 0b10101010)): # cr1: 32 bit
    qc.x(0)
with qc.if_test((cr2, 0b01010101)): # cr2: 32 bit
    qc.x(1)
# → 合計 64 bit のブロードキャスト → 60 bit 制限超え → エラー ❌
# 修正: barrier() で broadcast を分割
qc.measure(range(32), cr1)
with qc.if_test((cr1, 0b10101010)): # broadcast 1: 32 bit ✅
    qc.x(0)
qc.barrier() # ← ここで broadcast 区切り
qc.measure(range(32, 64), cr2)
with qc.if_test((cr2, 0b01010101)): # broadcast 2: 32 bit ✅
    qc.x(1)
```

Section3: Create quantum circuits

TASK 3.2: Construct parameterized circuits (例題近日追加予定)

REFERENCES:

<https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.Parameter>

<https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.QuantumCircuit>

<https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.ParameterExpression>

TASK 3.2: Overview

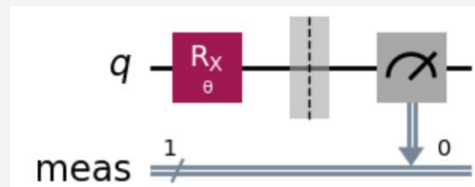
Parameterized circuit

量子回路のゲート角度などの値をシンボリックなパラメーターとして定義し、後から具体的な数値をバインド (代入) できる量子回路。同じ回路を異なるパラメーター値で繰り返し実行できる

- 用途: 変分量子アルゴリズム (VQE・QAOAなど)、量子機械学習など
- 作成

```
# パラメーター定義
theta = Parameter('theta')

# 回路にパラメーターを使用
qc = QuantumCircuit(1)
qc.rx(theta, 0) # Rx ゲートの角度をパラメータに
qc.measure_all()
```



- パラメーターのバインド
 - assign_parameters メソッド (bind_parameters メソッドは非推奨)

```
bound_qc = qc.assign_parameters({theta: np.pi / 2})
```

- Primitive の run メソッドで指定

```
job = sampler.run([(qc, param_values)])
```

- 複数パラメーターを一括管理

```
params = ParameterVector('theta', 4)
```

7. Given the code fragment below, which of the following code fragments creates a rotation gate with an angle with an initially undefined value?

```
from qiskit.circuit import QuantumCircuit, Parameter,  
    ParameterExpression  
qc = QuantumCircuit(1)
```

- a. `theta = 3.14`
`qc.rx(3.14, 0)`
- b. `theta = Parameter('theta')`
`qc.rx(theta, 0)`
- c. `qc.rx('theta', 0)`
- d. `qc.rx(ParameterExpression('theta'), 0)`

TASK 3.2: 例題 (from [sample test](#))

7. Given the code fragment below, which of the following code fragments creates a rotation gate with an angle with an initially undefined value?

```
from qiskit.circuit import QuantumCircuit, Parameter,  
    ParameterExpression  
qc = QuantumCircuit(1)
```

a. `theta = 3.14`

`qc.rx(3.14, 0)`

b. `theta = Parameter('theta')`

`qc.rx(theta, 0)`

c. `qc.rx('theta', 0)`

d. `qc.rx(ParameterExpression('theta'), 0)`

Section3: Create quantum circuits

TASK 3.3: Transpile and optimize circuits

REFERENCES:

<https://quantum.cloud.ibm.com/docs/en/guides/transpiler-stages>

<https://quantum.cloud.ibm.com/docs/en/guides/transpile-with-pass-managers>

https://quantum.cloud.ibm.com/docs/en/api/qiskit/transpiler_passes

TASK 3.3: Overview

Transpileの必要性

- 仮想 -> 物理量子ビットへのマッピング
- トポロジーによるSWAPゲート挿入
- 実機がサポートするゲートセット (Ex. cx, rz, sx, id) への変換
- 最適化: 回路の深さ・ゲート数減少によるエラー最小化

Transpile Pipeline

1. Init 独自の初期最適化 多量子ゲート分解	2. Layout 仮想->物理量子 ビットマッピング	3. Routing SWAPゲート挿入	4. Translation ネイティブゲート へ変換	5 Optimization ゲート削減・再合 成	6. Scheduling アイドル時間管理 (オプション)
--	--	--------------------------------	--	--	---

Transpileの方法

- transpile(circuit, backend): シンプル。内部で下記を呼ぶ
- generate_preset_pass_manager() + .run(circuit) — 推奨。

```
from qiskit.transpiler import generate_preset_pass_manager

pm = generate_preset_pass_manager(
    optimization_level=3, backend=backend
)
transpiled = pm.run(circuit)
```

Optimization Level

Level	Layout で使用するパス	Optimization ステージの主なパス	用途
Level 0	TrivialLayout	最小限のみ	速度優先・デバッグ
Level 1	TrivialLayout → VF2Layout → SabreLayout	Optimize1qGatesDecomposition + CXCancellation	バランス（推奨）
Level 2	VF2Layout → SabreLayout	Optimize1qGatesDecomposition + CommutativeCancellation	高品質
Level 3	VF2Layout → SabreLayout	Collect2qBlocks + ConsolidateBlocks + UnitarySynthesis + CommutativeCancellation	本番・最高品質



TASK 3.3: Overview

各パイプライン概要

1. Init

デフォルトでは何もしない。特殊な初期最適化や多量子ビットゲート変換。

2. Layout — 仮想→物理量子ビットマッピング

- **TrivialLayout**: 各仮想量子ビットを、デバイス上の同じ番号の物理量子ビットに単純にマッピング
- **VF2Layout**: 完全マッピング探索。部分グラフ同型問題として解く。複数解があれば誤り率スコアで最良を選択
- **SabreLayout**: ヒューリスティック。ランダム初期配置 + SabreSwap の反復で探索。SWAP 数の最小化が主目的。接続性の高いサブグラフを優先して選択

3. Routing — SWAP ゲート挿入

最適 SWAP 探索は NP 困難 — 全探索は実用上不可能。SabreSwap による 確率的ヒューリスティック手法をとる。実行のたびに結果が変わる

4. Translation — ネイティブゲートへ変換

- SWAP = 3 CNOT: ネイティブでない場合は 3 つの CX に分解
- Toffoli = 最大 6 CNOT: 3量子ビットゲートは 1Q/2Q ゲートに分解
- Translation で増えたゲートを次の Optimization ステージで削減

5. Optimization — ゲート削減・再合成

- **Lv1**: 単一量子ビットゲートの連鎖をまとめ、連続する CNOT をキャンセル
- **Lv2**: 交換関係を利用した冗長ゲート除去 (CommutativeCancellation)
- **Lv3**: 2量子ビットブロックを収集・統合し UnitarySynthesis で再合成。最大の削減効果

6. Scheduling — アイドル時間管理 (オプション)

- `scheduling_method="asap"` または `"alap"` を指定して有効化
- `ALAPSchedulingAnalysis` が各命令の開始時刻を計算し `Delay` 命令を挿入
- `PadDynamicalDecoupling` と組み合わせてエラー抑制に活用

PassManagerのカスタマイズ

- プリセットPMのステージ差し替え

```
pm = generate_preset_pass_manager(
    optimization_level=3, backend=backend
)
# pre_layout ステージを差し替え
pm.pre_layout = PassManager([
    InverseCancellation([
        HGate(), (TGate(), TdgGate()), ])]])
```

- カスタムPMの作成

ベストプラクティス

- `optimization_level` の選び方: プロトタイプは Level 1、本番は Level 3。最適化が強いほど実行コストも増加するため用途に応じて選択
- 確率的 Routing への対策: `seed_transpiler` を固定して再現性を確保。または複数シードで実行し品質の高い回路を採用
- 回路品質の確認方法: `circ.depth()` / `circ.count_ops()` で深さ・ゲート数を定量比較。CX ゲート数がノイズ耐性の目安

1. Which of the following lists the preset transpiler pipeline stages in the correct order?

- a. layout → routing → translation → optimization → scheduling → init
- b. init → layout → routing → translation → optimization → scheduling
- c. init → routing → layout → translation → scheduling → optimization
- d. layout → translation → routing → optimization → init → scheduling

TASK 3.3: 例題

1. Which of the following lists the preset transpiler pipeline stages in the correct order?

- a. layout → routing → translation → optimization → scheduling → init
- ☒ b. init → layout → routing → translation → optimization → scheduling
- c. init → routing → layout → translation → scheduling → optimization
- d. layout → translation → routing → optimization → init → scheduling

TASK 3.3: 例題 (from [sample test](#))

30秒

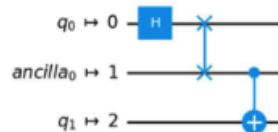
9. Given the code fragment below, which one of the following images could be produced?

```
from qiskit import QuantumCircuit
from qiskit import generate_preset_pass_manager
```

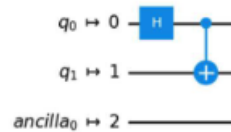
```
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0,1)
```

```
pass_manager = generate_preset_pass_manager(
    optimization_level=3,
    coupling_map=[[0, 1], [1, 2]] ,
    basis_gates=['h', 'swap', 'cx'],
    initial_layout=[0, 2]
)
tqc = pass_manager.run(qc)
tqc.draw(output="mpl")
```

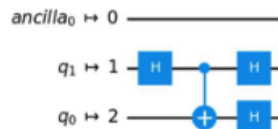
a.



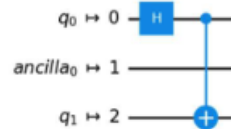
b.



c.



d.



TASK 3.3: 例題 (from [sample test](#))

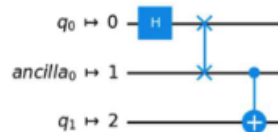
9. Given the code fragment below, which one of the following images could be produced?

```
from qiskit import QuantumCircuit
from qiskit import generate_preset_pass_manager
```

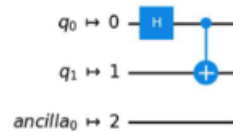
```
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0,1)
```

```
pass_manager = generate_preset_pass_manager(
    optimization_level=3,
    coupling_map=[[0, 1], [1, 2]] ,
    basis_gates=['h', 'swap', 'cx'],
    initial_layout=[0, 2]
)
tqc = pass_manager.run(qc)
tqc.draw(output="mpl")
```

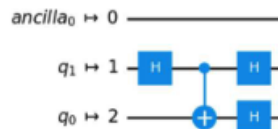
a.



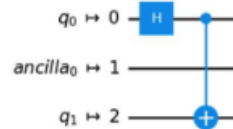
b.



c.



d.



Section3: Create quantum circuits

TASK 3.4: Construct basic quantum circuits

REFERENCES:

<https://quantum.cloud.ibm.com/docs/en/api/qiskit/circuit>

<https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.QuantumCircuit>



Much better reference?

<https://quantum.cloud.ibm.com/learning/en/courses/basics-of-quantum-information>

Basics of quantum
information

Overview

Lessons

Single systems

Multiple systems

Quantum circuits

Entanglement in action

Exam

Overview



Welcome to *Basics of quantum information*, the first course in the *Understanding quantum information and computation* series comprising the following courses:

Section4: Run quantum circuits

TASK 4.1: Demonstrate an understanding of execution modes such as: session with dedicated, priority, and batch mode

REFERENCES:

<https://quantum.cloud.ibm.com/docs/en/guides/execute-on-hardware>

<https://quantum.cloud.ibm.com/docs/en/guides/execution-modes>

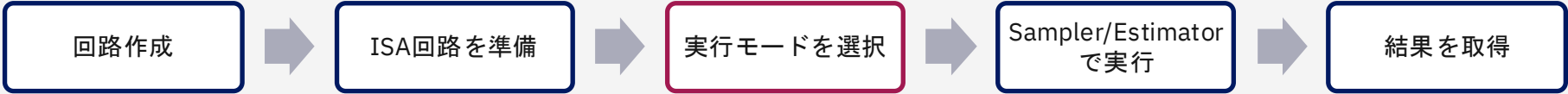
<https://quantum.cloud.ibm.com/docs/en/guides/choose-execution-mode>

<https://quantum.cloud.ibm.com/docs/en/guides/run-jobs-batch>



TASK 4.1: Overview

実行モードはジョブのスケジューリング方法を決定する。目的に応じて選ぶことで、コストと実行時間を最適化できる。



	Job	Batch	Session
概要	コンテキストマネージャ不要の単発リクエスト。	独立した複数ジョブを一括スケジューリング。古典処理を並列化してから QPU へ連続投入。	マルチジョブを実行するための専用ウィンドウ。QPUを専有。キャリブレーションも排除
用途	小規模な実験・単一ジョブ	全ジョブが事前に揃っている並列実験	VQE 等の反復アルゴリズム・ノイズモデル実験
指定方法	mode=backend	mode=batch or コンテキストマネージャ	mode=session or コンテキストマネージャ
Job数	1	複数 (独立)	複数 (反復可)
QPU占有			X
古典処理の並列化		X	X
2 job目以降のキュー待ち	あり	なし	なし
相対コスト	低	中	高
Open Plan	X	X	

TASK 4.1: Overview

TTL (Time To Live)

- **Maximum TTL:** Batch / Session 全体の最大有効時間。超過すると未実行ジョブはキャンセル。有料プランは最大 8 時間、Open は 10 分。
- **Interactive TTL:** 次のジョブが投入されるまでの待機上限 (全プラン 1 分)。超過するとワークロードは一時停止。

Batch modeの使い方

```
batch = Batch(backend=backend)
estimator = Estimator(mode=batch)
sampler = Sampler(mode=batch)
# Close the batch because no context manager was used.
batch.close() <- 新規受付停止、実行中/キュー待ちは完了
```

Context Manager (推奨)

```
with Batch(backend=backend):
    estimator = Estimator()
    sampler = Sampler()
```

Maximum TTLの変更

```
with Batch(backend=backend, max_time='25m'):
```

Batchの詳細

```
batch.details()
```

ベストプラクティス

- QPU 時間が 1分未満のジョブは複数まとめて1ジョブにする (固定オーバーヘッドを削減)。
- Session の専有アクセスが不要なら Batch を優先。
- Session を使う場合、大きなジョブを小さく分割して並列実行するとクロック時間を短縮できる。
- **一般的な推奨: まず Batch を検討し、反復処理や QPU 専有が必要な場合のみ Session を選ぶ。**

Session modeの使い方

```
session = Session(backend=backend)
estimator = Estimator(mode=session)
sampler = Sampler(mode=session)
# Close the session because no context manager was used.
session.close() <- 新規受付停止、実行中/キュー待ちは完了
```

Context Manager (推奨)

```
with Session(backend=backend):
    estimator = Estimator()
    sampler = Sampler()
```

Maximum TTLの変更

```
with Session(backend=backend, max_time='25m'):
```

Sessionの詳細

```
session.details()
```

6. Which statement about plan restrictions and best practices is correct?

- a. Open Plan users can freely run session jobs; Premium users cannot.
- b. Open Plan users cannot submit session jobs; generally, prefer Batch for multi-job workloads unless you need the added benefits of a Session.
- c. Batch mode is unavailable on paid plans.
- d. Job mode is deprecated and should be avoided.

TASK 4.1: 例題

6. Which statement about plan restrictions and best practices is correct?

- a. Open Plan users can freely run session jobs; Premium users cannot.
- ☒ b. Open Plan users cannot submit session jobs; generally, prefer Batch for multi-job workloads unless you need the added benefits of a Session.
- c. Batch mode is unavailable on paid plans.
- d. Job mode is deprecated and should be avoided.

Section4: Run quantum circuits

TASK 4.2: Demonstrate understanding of how to run quantum circuits with real hardware using Qiskit Runtime primitives and applying broadcasting rules

REFERENCES:

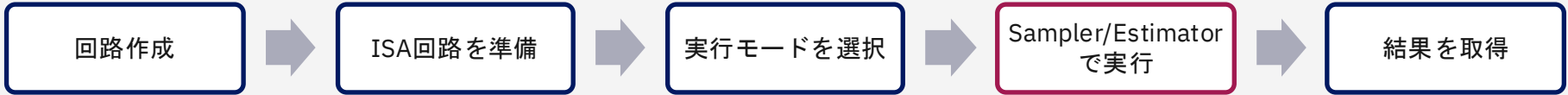
<https://quantum.cloud.ibm.com/docs/en/guides/execute-on-hardware>

<https://quantum.cloud.ibm.com/docs/en/guides/primitives>

<https://quantum.cloud.ibm.com/docs/en/guides/primitive-input-output>

<https://quantum.cloud.ibm.com/docs/en/guides/run-jobs-session>

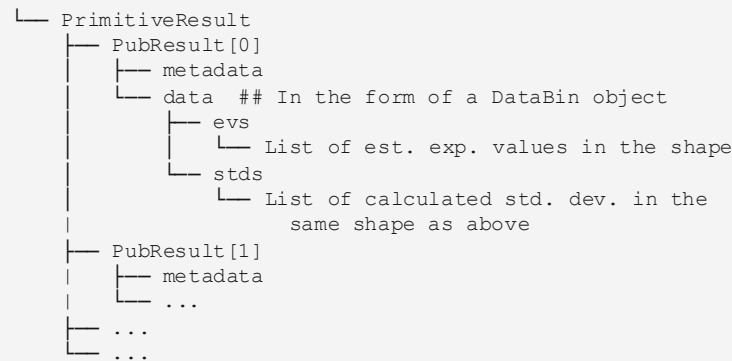
TASK 4.2: Overview



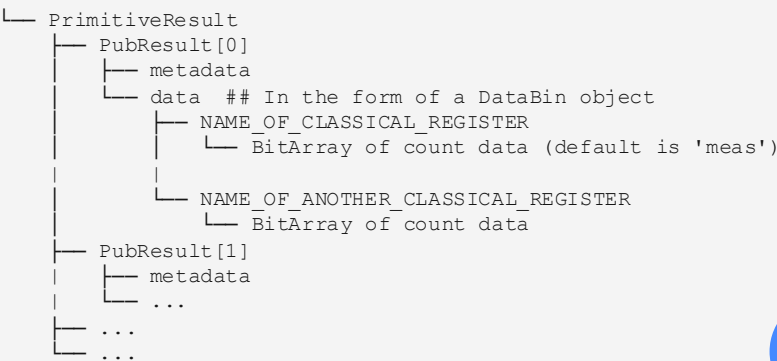
	Estimator	Sampler
用途	観測量の期待値	量子状態のサンプリング
入力 (PUB*タプル)	(circuit, observables, [params], [precision])	(circuit, [params], [shots])
出力	期待値 (evs) ・ 標準偏差 (stds)	ビット列 (BitArray)
典型的なユースケース	VQE, 相関関数	状態トモグラフィ, 確率分布

*...Primitive Unified Bloc

Estimatorの出力構造



Samplerの出力構造



TASK 4.2: Overview

Broadcastingルール

Primitiveの結果のShapeは、 observables と parameter_values の配列の組み合わせで決定。組み合わせは NumPy と同じBroadcastingルールに従う。

原則	説明	例
1. 次元補完	次元数が少ない側の 左端に 1 を補完	(5,) と (3, 5) → (1, 5) と (3, 5)
2. 右端から比較	サイズの比較は 右端の次元から 左に向かって行う	(11,2,7) と (11,1,7) → OK
3. 互換条件	サイズが等しい or どちらかが 1	(5,) と (3,) → 非互換

4つの典型パターン

Pattern 1 — Single Broadcast (単一Observable)

params (5,)+obs ()→(5,)

1つのObservableを全パラメータに適用

Pattern 2 — Zip (1対1対応)

params (5,)+obs (5,)→(5,)

n番目のparamとn番目のobsを対応させる

Pattern 3 — Outer/Product (全組み合わせ)

params (1,6)+obs (4,1)→(4,6)

4つのObsと6つのparamの全24通りを一括計算

Pattern 4 — nd一般化 (多次元)

params (3,6)+obs (2,3,1)→(2,3,6)

Q 高次元ベクトル化でバッチ計算

注意

- 回路中のParameterの個数はBroadcastの対象外なので取り除く
- SparsePauliOp の shape:
 - SparsePauliOp.from_list(["XX","XY","IZ"]) は shape () (スカラー)
 - [SparsePauliOp("XX"), SparsePauliOp("XY"), SparsePauliOp("IZ")] は shape (3,)



3. A circuit has two **Parameters**. You pass parameter bindings shaped $(100, 2)$ and an observables array shaped $(3, 1)$ to **EstimatorV2** in one PUB. What is the shape of the resulting expectation value array for that PUB?

- a. $(3, 100)$
- b. $(100, 3)$
- c. $(3,)$
- d. $(100,)$

TASK 4.2: 例題

3. A circuit has two **Parameters**. You pass parameter bindings shaped **(100, 2)** and an observables array shaped **(3, 1)** to **EstimatorV2** in one PUB. What is the shape of the resulting expectation value array for that PUB?

- a. (3, 100)
- b. (100, 3)
- c. (3,)
- d. (100,)

【前提】 params の shape から「パラメータ個数」の次元を除く

配列	元の shape	broadcast 対象の shape	理由
parameter_values	(100, 2)	(100,)	最終次元 2 = パラメータ個数 → 除外
observables	(3, 1)	(3, 1)	observable 配列はそのまま

【ステップ1】次元数を揃える（左に 1 を補う）

配列	補う前	補った後（2次元に揃える）
params (broadcast 対象)	(100,)	(1, 100) ← 左に 1 を補った
observables	(3, 1)	(3, 1) ← 変わらない

【ステップ2】右端から各次元を比較

次元（右から）	params	obs	比較結果	出力
右端（第2次元）	100	1	どちらかが 1 → OK	100（大きい方）
左（第1次元）	1	3	どちらかが 1 → OK	3（大きい方）

【ステップ3】結果の shape を確定

結果 shape = (3, 100)
= 「3種類の observable」 × 「100セットのパラメータ」の期待値が格納される配列

END